

Matter and antimatter, when brought together, create the most energetic reaction known to science. Uncontrolled, this reaction could cause devastation. If regulated, however – such as in the core of a warp drive – the resulting power source could change the world.

So too is it true with UI and application logic. When intermingled carelessly, application code is bloated, tangled, difficult and sometimes impossible to maintain. When separated carefully, connected only by the narrowest of communications channels, elegant, dynamic, and highly adaptable software can be built, and in record time.

Frameworks like MVC and WPF (using the MVVM pattern) pioneered this approach. Antimatter takes it to the next level. Application logic and UI are separated so cleanly in Antimatter applications that either layer can literally be constructed using nearly any language or framework, and run in different CPU processes or even different machines all together.

## The Elements of an Antimatter Application

Every Antimatter application has two layers provided by the application, and one provided by the framework.

### Application-Defined Layers:

- 1) **The Model.** The business logic of the application is defined by a platform-independent Model. A **Root** serves as the entry-point for the application, to which any number of child **View Models** are accessed through properties. View Models expose the properties – text and other primitive data – to which UI elements are linked through **Bindings**, and update those properties (and thus the UI) as needed. View Models also can expose **Command** properties, which are invoked by UI elements in response to user input. The execution of Commands, in turn, typically results in changes to the Model, which are communicated to the UI.
- 2) **The UI.** The UI layer contains everything necessary to visualize the primitive data elements exposed by the Model, and is responsible for processing user input by invoking Commands on the Model. Anything platform-dependent is also handled in the UI layer.

### Framework-Defined Layers:

- 3) **The Reactor.** The Antimatter framework provides a component we call the Reactor (as in, the matter/antimatter reactor). Portions of the Reactor exist on both the Model and UI sides of the application, allowing each layer to exchange data with the other. Specifically, the Reactor manages Bindings between UI elements and Model properties by listening for Model changes and notifying bound UI elements of them, and by notifying View Models of the invocation of Commands by the bound UI element in response to input.

## Antimatter in Practice

The first incarnation of the Antimatter framework supports .NET-based Models and web-based UIs constructed with React and Typescript (technically plain Javascript is supported, but the developer experience is far more optimal with Typescript for reasons we'll see). Again, virtually any language/framework can be used for either layer once additional Reactors have been developed. Below we'll explore both the Model and UI sides of a simple HR management (what else?) application.

## The Model Layer

A Model constructed in .NET/C# will look familiar – really identical – to what one would create in a XAML-based framework when using the MVVM (model-view-viewmodel) pattern. We don't call it MVVM, because Antimatter doesn't care if the data layer (the "model" in MVVM) is distinct from the view model layer – that decision lives entirely on the Model side of the matter/antimatter barrier, and so the Antimatter framework is agnostic to it. It's all just "Model" as far as Antimatter is concerned.

The key to successfully building a platform-agnostic Model is to identify the logical components of the application and how they all flow from a single root object, and build those objects accordingly. In our HR app example – and as is a best practice for Antimatter generally – we start with a single root ApplicationViewModel, from which all our other model components are eventually accessed:

```
public class ApplicationViewModel : ObservableObject
{
    public CompanyViewModel Company { get; } = new CompanyViewModel();
}

public class CompanyViewModel : ObservableObject
{
    public string Name
    {
        get => GetValue<string>("Contoso");
        set => SetValue(value);
    }

    public ObservableCollection<EmployeeViewModel> Employees { get; } =
        new ObservableCollection<EmployeeViewModel>();
}

public class EmployeeViewModel : ObservableObject
{
    public string FirstName
    {
        get => GetValue<string>();
        set => SetValue(value);
    }

    public string LastName
    {
        get => GetValue<string>();
        set => SetValue(value);
    }

    public string Title
    {
        get => GetValue<string>("Associate");
        set => SetValue(value);
    }

    public double Salary
    {
        get => GetValue<double>(10000);
        set => SetValue(value);
    }
}
```

Each of these ViewModels inherits from the same ObservableObject base class; while this is by no means necessary to build an Antimatter Model, what *is* necessary (in C# Models at least) is that a Model Component implement the INotifyPropertyChanged interface and invoke the PropertyChanged event whenever there is a change to a component property which might need to be reflected in the UI. Here is our “reference” implementation of ObservableObject:

```
public class ObservableObject : INotifyPropertyChanged
{
    private Dictionary<string, object> _properties = new Dictionary<string,
object>();

    public event PropertyChangedEventHandler PropertyChanged;

    protected T GetValue<T>(T defaultValue = default, [CallerMemberName] string
property = null)
    {
        if (!_properties.TryGetValue(property, out object value) ||
            !(value is T typedValue))
            return defaultValue;
        return typedValue;
    }

    protected void SetValue(object value, [CallerMemberName] string property = null)
    {
        _properties[property] = value;
        OnPropertyChanged(property);
    }

    protected void OnPropertyChanged([CallerMemberName] string property = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(property));
    }
}
```

This ObservableObject example makes implementations extremely easy, albeit with a modest performance penalty for using a property bucket dictionary to hold values rather than backing fields. Most importantly, all property setters in our sub-classes call SetValue, which ultimately invokes the PropertyChanged event to notify subscribers – in this case the Antimatter Reactor – of a property change that should be communicated to bound UI elements.

Again, however, there is no requirement that you use this implementation of ObservableObject (the Antimatter.NET core library does include it, but you may have good reasons not to want to make your Model assemblies dependent on Antimatter.NET); implementing INotifyPropertyChanged, and making use of related classes like ObservableCollection, are all that is required – in other words, any MVVM-compatible view model code can be used with Antimatter.NET.

### The UI Layer

Just as Antimatter currently supports Models constructed in .NET, the first (of hopefully many) supported platform on the UI side web applications built with React and Typescript, using the @antimatterjs/react npm package. Let’s take a look at a barebones UI layer built in React and Antimatter.JS that could be used in connection with the HR app we modelled above. In this example,

we'll be using React class components primarily. As we'll see below, it is definitely possible to use React function components with Antimatter if that's your preference, although it requires using a component framework – such as Positron – that is built to work with Antimatter for any UI elements to which you wish to bind Model values.

Without using any additional component library, however, we can construct a very simple UI for our HR app, such as the following: